Taking Dual Shortcut

Primal (Max)	Dual (min)
ith con $\leq$	ith var $\geq$
ith con $\geq$	ith var $\leq$
ith con $=$	ith var unrestricted
jth var $\geq$	jth con $\geq$
jth var $\leq$	jth con $\leq$
jth var unrestricted	jth con $=$

Note that the costs of the primal are now the b of the dual and that the b of the primal are now the costs of the dual.

When trying to figure out how taking the dual of parametric LP's think about when a variable appears in more than one constraint, in which case those constraints are all important for that variable in the dual. If a variable x appears in constraints $a_1, ..., a_l$ then those constraint variables $a_1, ..., a_l$ appear in the dual in the constraint corresponding to the variable x. Addtionally, these constraint variables appear in this specific constraint with the same scalar multiple in which x appeared in the individual constraints.

Lagrangian Relaxtion in QUBO

$L(\lambda) = \min c^T x - \lambda(f^T x - g)$

Boolean Condition	Linear Constraint	Quadratic Constraints
$x \vee y$	$x + y \geq 1$	$1 - x - y + xy = 0$
$x \to y$	$x \leq y$	$x - xy = 0$
$\neg(x \wedge y)$	$x + y \leq 1$	$xy = 0$
$x \wedge y$	$x + y = 2$	$1 - xy = 0$
$x \oplus y$	$x + y = 1$	$(1 - x - y)^2 = 1 - x - y + 2xy$
$\oplus(x)$	$\sum x = 1$	$(1 - \sum x)^2 \equiv 1 - \sum_i x_i + 2 \sum_{0 \leq i < j \leq n} x_i x_j$

Finding Basic Feasible Solutions

A bfs $\bar{x}$ is found by finding a basis B of A and inverting B then multiplying by b to get the vector x where any variables not in the basis b are simply set to 0.

π & c^π

For a basis B of A:

$$\pi = (B^{-1})^T C_B$$
$$C^\pi = C - A^T \pi$$

Proving Optimality

Either show that $c^\pi \geq 0$ or that complementary slackness is 0 ie $y(Ax - b) = x(A^T y - c) = 0$. Note complementary slackness only applies to non equality constraints as the complementary slackness would be 0 for any feasible solution.

Runtime Stuff

Given n integers in the range $[0, N-1]$

$O(n + N)$ time, # bits to represent the input

Since input is in $[0, N-1]$, that means the input size is actually $\Theta(\log N)$ so runtime is actually $\Theta(n \log N)$

In other words, if the runtime is $O(N)$ then this is actually exponential time because the input size is $\Theta(\log N)$

MCNF Definitions

$\min \sum_{i,j \in E} c_{ij} x_{ij}$

s.t. $\sum_{j:(i,j)\in E} x_{ij} - \sum_{j:(j,i)\in E} x_j i = b_i, (i \in V)$ (mass balance)

$0 \leq l_{ij} \leq x_{ij} \leq u_{ij}, ((i,j) \in E)$ (capacity)

Note that the node arc incidence matrix is totally unimodular when there is no upper or lower bounts, which can be removed by transformations.

Note that a column of the node arc incidence has a single -1 and 1 entry corresponding to the nodes

- Optimal Sol Guranteed Integer
- Basic solution of $Ax = b$ (b is integer) are integer

- B^{-1} integer $\forall$ basis B of A

Def A is totally unimodular iff $\forall$ square submatrix F of A, $det(F) \in \{\pm 1, 0\}$

Duality of MCNF

$\max \sum_{i \in V} b_i \pi_i - \sum_{(i,j) \in U} u_{ij} \alpha_{ij}$

s.t. $\pi_i - \pi_j \leq c_{ij}$ $((i,j) \notin U)$ this can be rewritten as $c_{ij}^{\pi} \geq 0$

$\pi_i - \pi_j + \alpha_{ij} \leq c_{ij} ((i,j) \in U)$ this can be rewritten as $c_{ij}^{\pi} + \alpha_{ij} \geq 0$

π_i unrestricted and $\alpha_{ij} \geq 0$

$x_{ij} c_{ij}^{\pi} = 0, ((i,j) \notin U)$

$x_{ij}(c_{ij} + \alpha_{ij})^{\pi} = 0, ((i,j) \in U)$

$\alpha_{ij}(u_{ij} - x_{ij}) = 0, ((i,j) \in U)$

MCNF Equivalent Modifications

1. No parallel arcs (beneficial if the arcs have different costs with different upperbounds)

   This can be solved by putting a fake node with a supply of 0 and one side having same costs

2. No symmetric arcs

   Similarly solved by adding a fake node again with a supply of 0 and one side having same costs

3. No lower bounds ($l_{ij} = 0$)

   We can add a middle node with a supply of $-l_{ij}$, the left arc would be the same (costs and upperbound) and so is the left node, the right node now has a supply of $b_j - l_{ij}$

   Alternatively, without creating a new node we can decrease the supply of the left node by l_{ij} and increase the supply of the right node by l_{ij} and decreasing the upperbound of the arc by l_{ij} (we can ignore the cost of sending l_{ij} because it is constant) (note this problem is equivalent but does not preserve the objective function of the original)

4. Cost reversal

   For any negative cost we can reverse the arc direction, take the negative of the cost and preserve the upperbound of the arc. The original left node's supply is subtracted by the upperbound and the right node's supply adds the upperbound. Essentially this is saying were assuming the maximum flow in the original direction and adding back the flow as needed

5. Removing Upper Bounds ($u_{ij} = \infty$)

   $x_{ij} \leq u_{ij} \rightarrow x_{ij} + s_{ij} = u_{ij}$, suppose we have an arc i, j with upperbound u_{ij}. We can create a dummy node in between nodes i, j with a supply of u_{ij}, subtract that supply from b_i, and the cost of going from node ij to j is c_{ij} and an arc from ij to i with 0 cost. Essentially we are removing the arc from i to j and enforcing the upper bound by creating a node with the upper bound as a supply and allowing that node to push out the supply back to i (deficit of the upperbound) or push to j same as taking that arc.

Resuidual Network

Suppose two nodes i, j with cost c_{ij} and a cost u_{ij} traversed with a flow x_{ij}

$G(V, E) + x \rightarrow G(X) = (V, E(x))$ which is basically updated version of edges given that flow exists already in the graph, IE the residual flows that can still be sent through the arcs. In this example our arc i, j has now an upper bound $u_{ij} - x_{ij}$. We also add an arc from j to i to essentially send back flow x_{ij} so this arc has an upperbound of x_{ij} with a cost of $-c_{ij}$. Note we do need to omit any arcs with upperbounds of 0